

Task 2

Розгортання і робота з distributed in-memory data structures на основі Hazelcast: Distributed Map (10 балів)

Hazelcast є розподіленою платформою для зберігання та обробки даних в реальному часі в оперативній пам'яті. "Розподілене" означає те, що на кожній з нод (серверів) системи запускається свій екземпляр Hazelcast, які потім об'єднуються в загальний кластер. В рамках цього кластера, через API можна створювати різні розподілені структури даних: *Map, Queue, Topic, Lock, Data-stream processing pipeline,...*

Таким чином, платформу Hazelcast можна застосовувати, як швидкий кеш в пам'яті, систему для розподілених обчислень, систему для обробки потоку повідомлень у режимі реального часу, ...

Запуск Hazelcast з Java-застосування

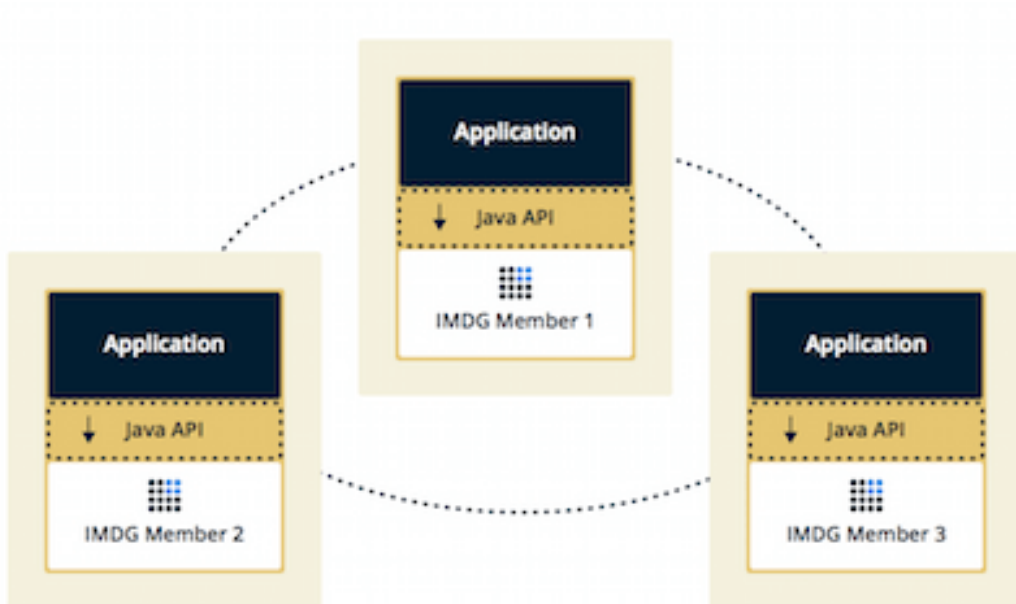
Ноду можна створювати та запускати напряму з Java-додатку, який буде працювати на тому же сервері де запусканий екземпляр Hazelcast: <https://docs.hazelcast.com/hazelcast/5.3/getting-started/get-started-java>

Додаток матиме доступ через API до цих розподілених структур даних, і зможе писати/читати в/з них.

```
HazelcastInstance hzInstance = Hazelcast.newHazelcastInstance();
Map<String, String> distributedMap = hzInstance.getMap(
    "capitals" );
capitalcities.put( "1", "Tokyo" );
capitalcities.put( "2", "Paris" );
```

При цьому, інші додатки підключені до інших нод кластеру Hazelcast будуть також бачити зміни в розподілених структурах даних, і також можуть писати/читати в/з них.

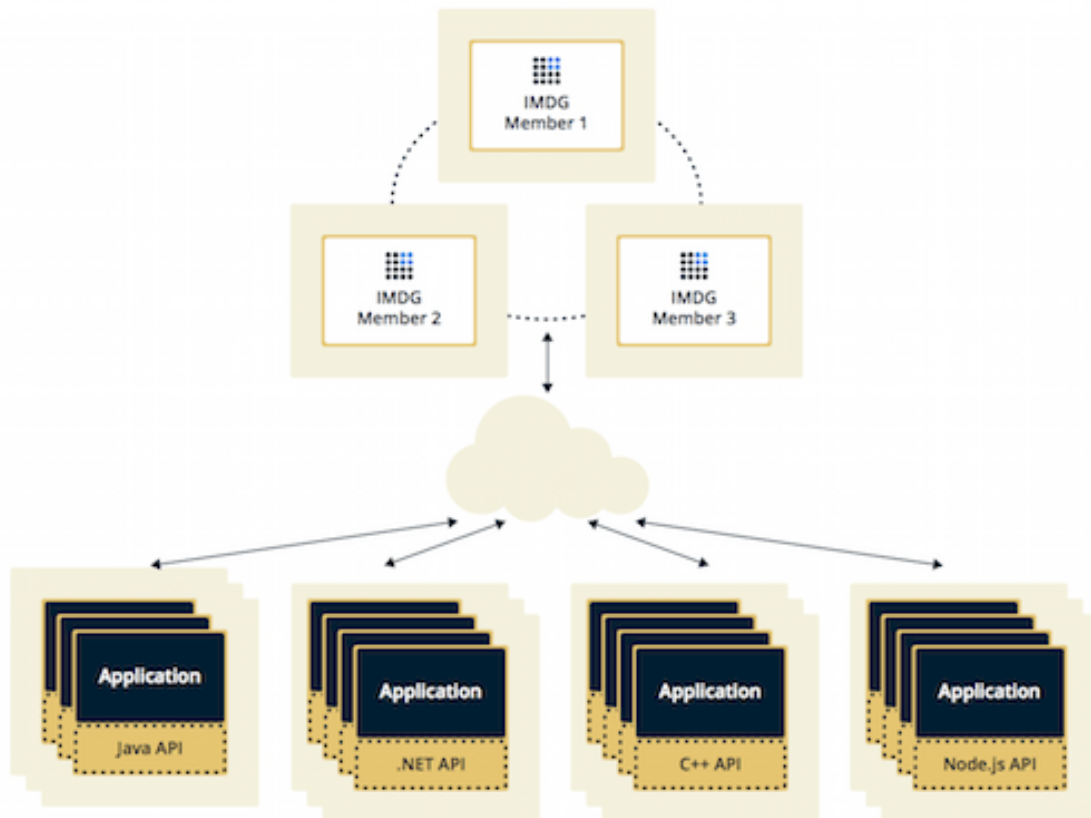
<https://docs.hazelcast.com/hazelcast/5.3/deploy/choosing-a-deployment-option>



Запуск нод Hazelcast окремо

Іншим способом є запуск нод кластеру, як окремих застосунків через командний рядок (<https://docs.hazelcast.com/hazelcast/5.3/getting-started/get-started-binary>) або як Docker-контейнер (<https://docs.hazelcast.com/hazelcast/5.3/getting-started/get-started-docker>) та підключення до нього за допомогою клієнтів, які існують під різні мови програмування

```
HazelcastInstance hzInstance = Hazelcast.newHazelcastClient();
Map<String, String> distributedMap = hzInstance.getMap(
    "capitals" );
capitalcities.put( "1", "Tokyo" );
capitalcities.put( "2", "Paris" );
```



Завдання:

- 1) Встановити і налаштувати Hazelcast
<https://hazelcast.com/open-source-projects/downloads/>
- 2) Сконфігурувати і запустити 3 ноди (інстанси) об'єднані в кластер або як частину Java-застосування, або як окремі застосування
<https://docs.hazelcast.com/hazelcast/5.3/getting-started/get-started-binary#step-6-scale-your-cluster>
- 3) Продемонструйте роботу Distributed Map
 - використовуючи API на будь-якій мові яка має клієнт для Hazelcast, створіть Distributed Map
 - запишіть в неї 1000 значень з ключами від 0 до 1000
 - за допомогою Management Center
(<https://docs.hazelcast.com/management-center/5.3/getting-started/install#before-you-begin>) подивитись на розподіл ключів по нодах
 - подивитись як зміниться розподіл даних по нодах:
 - якщо відключити одну ноду (результати мають бути у протоколі)

- відключити послідовно дві ноди (результати мають бути у протоколі)
 - відключити одночасно дві ноди (емулюючи “падіння” серверів, чи використовуючи команду `kill -9`) (результати мають бути у протоколі)
 - Чи буде втрата даних?
 - Яким чином зробити щоб не було втрати даних?
- 4) Продемонструйте роботу Distributed Map without locks
- використовуючи 3 клієнта, на кожному з них одночасно запустіть інкремент значення для одного й того самого ключа в циклі на 10K ітерацій:
- ```
map.putIfAbsent("key", 0);
for (int k = 0; k < 10_000; k++) {
 var value = map.get("key");
 value++;
 map.put("key", value);
}
```
- подивиться яке кінцеве значення для ключа “key” буде отримано (чи вийде 30K?)
- 5) Зробіть те саме з використанням песимістичним блокування та поміряйте час:  
<http://docs.hazelcast.org/docs/latest/manual/html-single/index.html#locking-maps>
- 6) Зробіть те саме з використанням оптимістичним блокуванням та поміряйте час::  
<http://docs.hazelcast.org/docs/latest/manual/html-single/index.html#locking-maps>
- 7) Порівняйте результати кожного з запусків
- для реалізації без блокувань маєте спостерігати втрату даних; для реалізації з песимістичним та оптимістичним блокуванням мають бути однакові результати
  - песимістичний чи оптимістичний підхід працює швидше?
- 8) Робота з Bounded queue
- на основі Distributed Queue  
<https://docs.hazelcast.com/hazelcast/5.3/data-structures/queue#creating-an-example-queue>) налаштуйте Bounded queue на 10 елементів  
<https://docs.hazelcast.com/hazelcast/5.3/data-structures/queue#configuring-queue>)
  - запустіть одного клієнта який буде писати в чергу значення 1..100, а двох інших які будуть читати з черги
    - під час вичитування, кожне повідомлення має вичитуватись одразу
  - яким чином будуть вичитуватись значення з черги двома клієнтами?
  - перевірте яка буде поведінка на запис якщо відсутнє читання, і черга заповнена

## Вимоги до протоколу

У протоколі має міститись:

- посилання на GitHub з кодом
- вміст консолі з виводом результатів виконання коду
- скріншоти, та повідомлення, на основі яких можна побачити/зрозуміти результати по кожному пункту завдання

Hazelcast Python Client <https://hazelcast.com/clients/python/>

Hazelcast Go Client <https://pkg.go.dev/github.com/hazelcast/hazelcast-go-client>